MCS 320 Project One : Algebraic Numbers with `QQbar`
due Wednesday 1 July 2026, at 2pm.

`QQbar` extends the field `QQ` of rational numbers with algebraic numbers. An algebraic number is defined as a root of polynomial with rational coefficients. To avoid the expression swell caused by exact computation, with roots, floating-point interval arithmetic is applied. Interval arithmetic is the computation with intervals. The intervals store the lower and upper bounds on the floating-point, number represented by the interval.

In this project, we use SageMath to explore algebraic numbers with `QQbar`. The project description is derived from a Jupyter notebook with SageMath kernel. The best way to start the project is to download and execute the notebook posted at the course web site.

1. Defining QQbar Numbers

The field `QQbar` extends the field `QQ` of rational numbers. Therefore, we need to start with a polynomial with rational coefficients. As the example, we take $p = x^5 - x - 1$. In the code below, the expression $x^5 - x - 1$ is cast into the polynomial ring `QQ[x]`, so that p is a polynomial with rational coefficients.

```
p = QQ[x](x^5 - x - 1)
```

As p has degree 5, it has 5 roots and we have to specify a small enough interval that isolates a root. Following the documentation, we take $[0.1, 0.2]$ as the interval for the real part and $[1.0, 1.1]$ for the interval of the imaginary part of the root. The `CIF` abbreviates `ComplexIntervalField` and `RIF` abbreviates `RealIntervalField`. In the code below the interval `iv` is constructed.

```
iv = CIF(RIF(0.1, 0.2), RIF(1.0, 1.1))
```

The `0.2?  + 1.1?*I` is the short output form of `iv`. The question marks in the short output form of `iv` indicate that we have at most once decimal place of accuracy in the interval number. With the long output form, as the result of `iv.str(style='brackets')` the `brackets` style shows the explicit lower and upper bounds for the complex interval.

Now we are ready to define `r` as a root of p, in the interval `iv`:

```
r = QQbar.polynomial_root(p, iv)
```

The `r in iv` returns `True`. The minimial polynomial of an algebraic number is the polynomial with smallest degree that has that number as a root. It is available in the method `minpoly()` on a element of `QQbar`. The *symbolic verification* of `r` as an algebraic number defined by p is by verifying that `r.minpoly()` returns p. In addition, the evaluation of p at `r`, done via `p(x=r)` and verifying that `0` is returned is the shorted form of a symbolic verification.

We may view `r` as an element of `CIF`, after a typecast:

```
CIF(r).str(style='brackets')
```

shows the representation of `r` as a complex interval. The *numerical verification* that `r` is a root of p applies complex interval arithmetic and is done by `p(x = CIF(r))` which returns a number close the double precision.

The fragile step in the construction of an element of `QQbar` is the selection of an initial complex interval in the constructor. Asking for all roots of the polynomial in `QQbar` facilitates the construction of this interval, done via `p.roots(QQbar)`. To compute the n-th roots of a number, there is a shortcut: `QQbar(2).nth_root(5, all=True)` returns the list of all 5-th roots of 2.

Assignment One. Define a complex fifth root of two, that is: $2^{1/5}$, as an element `r` of `QQbar`.

1. Verify the construction of `r` symbolically.
2. Verify the construction of `r` numerically.

Write at least one sentence concluding each of your computations.

2. QQbar Arithmetic

Because we defined `r` as a root of the polynomial $x^5 - x - 1$, we have that $r^5 - r - 1 = 0$, or equivalently: r^5 simplifies into $r + 1$. Let us verify this property, in the code below:

```
z1 = r^5
z2 = r + 1
z1 == z2
```

which returns `True`. However:

```
CIF(z1) == CIF(z2)
```

returns `False`, because although as algebraic numbers r^5 and $r + 1$ are the same, their numerical representations as complex intervals are not the same.

Let us compare with the exact way of working with algebraic numbers:

```
E.<a> = QQ.extension(p)
```

defines the field E as $\mathbb{Q}/(a^5 - a - 1)$, as the extension of the rational numbers $\mathbb{Q}$ using a root of the polynomial p, denoting this root by a. Then computing a^5 results in the exact symbolic expression $a + 1$.

The numbers in the field E are polynomials in a of degree 4 or less with rational coefficients, whereas in the field `QQbar`, the numerical representations of the numbers are complex intervals, which are more compact, and therefore, the arithmetic goes faster. With `timeit('a**42')`, we measure the time it takes to evaluate a^{42} and we can compare this against `timeit('r**42')`. The next assignment invites to extend this comparison.

Assignment Two. In addition to `r` in `QQbar` as a root of $p = x^5 - x - 1$, define `s` in `QQbar` as a root of

$$q = x^5 - x - \frac{100001}{100000}.$$

To compare with the exact extensions, extend $E = \mathbb{Q}[a]/(a^5 - a - 1)$ with b as a root of q in to F, that is:

$$F = (\mathbb{Q}[a]/(a^5 - a - 1))[b] \Big/ \left(b^5 - b - \frac{100001}{100000}\right).$$

3. Coordinates of Regular Polygons

The complex roots of unity lie in the complex plane on a circle with radius one and define the corners of a regular polygon. Consider the pentagon.

```
pentagon = QQbar(1).nth_root(5, all=True)
p1 = [cos(RR(2*pi/5)), sin(RR(2*pi/5))]
```

The point `pentagon[1]` has coordinates $(\cos(2\pi/5), \sin(2\pi/5))$ as confirmed by `p1`. The exact representation of this first point is defined by its minimal polynomial:

```
(pentagon[1].real().minpoly(), pentagon[1].imag().minpoly())
```

which shows the minimial polynomial for the real and imaginary parts of the complex number representation of the first corner of the pentagon.

Assignment Three. Consider an increasing sequence of values for n, starting at $n = 6$, and construct the minimal polynomial for the coordinates of a regular n-gon.

1. Describe the relationship between n and the degree of minimal polynomials.
2. Time the construction of minimal polynomial. For each increase in n, how does the time increase?

4. The Deadline is Wednesday 1 July, at 2pm.

Upload your answer to gradescope at the latest on Wednesday 1 July, before 2pm.

The solution consists of one single notebook, organized according to the assignments. Mark the start of each solution to an assignment using a heading in a cell. Your notebook should run from top to bottom as a program without errors. Apply proper formatting in your notebook so it reads like a technical report if you would print it. Document the execution cells with complete sentences, properly formatted in markdown cells.

You may (not must) work in pairs for this project. A pair consists of two, not three or more. If you decide to work in a pair, then you must send me an email with the name of your partner and with the email address of your partner in the copy of the email, before 2pm on Wednesday 24 June. If working in a pair, then only one Jupyter notebook should be submitted.

If you have questions, concerns, or difficulties, feel free to contact me for help.